

Итоговый отчет и Рефлексия (Final Report)

1. Определение документа

Итоговый отчет (в индустрии часто называется Post-mortem или Sprint Review) — это аналитический документ, который подводит итоги всего процесса разработки. Проект редко идет строго по изначальному плану. Этот документ — честный разговор о том, что удалось реализовать, от каких функций пришлось отказаться (технический долг), с какими инженерными вызовами столкнулась команда и какие выводы были сделаны.

2. Назначение документа

Данный документ используется для:

- Оценки того, достиг ли проект стадии MVP (Minimum Viable Product — минимально жизнеспособный продукт).
- Анализа ошибок в оценке сроков и сложности задач.
- Фиксации уникальных технических решений (чтобы в будущем не изобретать велосипед).
- Прозрачного оценивания индивидуального вклада каждого участника, если проект выполнялся в команде.
- Планирования дальнейшего развития продукта (Roadmap).

3. Структура документа

- **Статус готовности и сравнение с ТЗ:** метрика выполнения изначальных требований. Списки реализованного функционала и функционала, который перенесен в статус "отложено". Причины отказа от функций.
- **Архитектурные и инженерные вызовы (Технические сложности):** самая важная часть. Глубокий разбор 1-2 самых сложных проблем, возникших в процессе кодирования. Описание симптомов проблемы, пути расследования и финального программного решения.
- **Организация рабочего процесса и Командная динамика:** какие инструменты управления проектами применялись (Trello, Jira), как велась работа с Git (использовался ли GitFlow), распределение ролей и зон ответственности.
- **Рефлексия и приобретенный опыт:** анализ новых изученных паттернов проектирования, фреймворков или алгоритмов. Оценка того, что команда сделала бы иначе, начиная проект заново.
- **План дальнейшего развития (Roadmap):** приоритизированный список функций (Backlog) для будущих версий продукта. (может быть в любом виде, не обязательно как в примере)

4. Критерии оценивания (Максимум: 10 баллов)

- [3 балла] **Глубина технической рефлексии:** детальность разбора инженерных проблем. Умение не просто написать "была ошибка базы данных, я погуглил и исправил", а объяснить физику проблемы и логику решения.
- [3 балла] **Объективность анализа результатов:** способность критически оценивать свою работу, обосновывать причины накопления технического долга (нехватки времени/знаний).
- [2 балла] **Качество описания процессов:** применение современных методологий организации работы (хотя бы базовое управление ветками Git и таск-трекерами).
- [2 балла] **Перспективное планирование:** технологическая и бизнес-обоснованность предложений по развитию системы.

5. Обязательные требования (Критические моменты)

В начале документа должен присутствовать однозначный вывод: достиг ли проект состояния MVP (готово ли оно к демонстрации пользователю).

Обязательно наличие развернутого технического разбора (Post-mortem) минимум одной сложной проблемы, потребовавшей рефакторинга кода или изменения архитектуры.

6. Пример документа (Система управления коворкингом)

1. Оценка готовности проекта (Достижение MVP)

Дорожная карта и статус

Готовность проекта (MVP)

Соотношение реализованного функционала к ТЗ

85%

Статус по ключевым модулям:

✓ API на FastAPI с авторизацией

Выполнено 100%

✓ Интерактивная карта на React

Выполнено 100%

✓ Алгоритм проверки пересечений (БД)

Выполнено 100%

✗ Интеграция с банковским API

Отложено (Mock)

✗ Динамическое ценообразование

Отложено

- **Общий статус:** Проект успешно достиг стадии MVP. Основной бизнес-процесс (выбор места -> бронирование -> оплата) работает стабильно. Уровень соответствия изначальному ТЗ: 85%.
- **Реализовано:**
 - Полноценный API на FastAPI с авторизацией по JWT-токенам.
 - Интерактивная карта на React, масштабирующаяся под мобильные устройства.
 - Сложная система проверки пересечений времени аренды рабочих мест.
- **Отложено в Технический долг:**
 - Интеграция с реальным банковским API (ЮKassa). Из-за сложности получения тестовых ключей на данный момент используется заглушка (Mock-объект), всегда возвращающая статус "Оплачено".
 - Динамическое ценообразование (повышение цен в часы пик). Логика оказалась слишком сложной для текущей архитектуры БД.

2. Инженерный вызов: Проблема "Гонки данных" (Race Condition)

- **Симптомы:** На этапе тестирования мы симулировали высокую нагрузку. Если два пользователя нажимали кнопку "Забронировать место #1 на 12:00" одновременно (с разницей в несколько миллисекунд), система одобряла обе заявки. Приложение делало запрос SELECT — видело, что место свободно для обоих, и оба запроса делали INSERT новой брони. Возникал конфликт (Овербукинг).
- **Исследование:** Проблема классифицирована как "Race Condition". Изначальная архитектура не учитывала атомарность операций.
- **Решение:** Мы полностью переписали алгоритм транзакций. Вместо обычного SELECT мы внедрили SQL-конструкцию SELECT ... FOR UPDATE. Теперь, когда

первая заявка проверяет место, база данных блокирует эту строку для чтения другими запросами до тех пор, пока первая заявка не завершит операцию записи. Вторая заявка встает в очередь ожидания и после разблокировки уже видит, что место занято, возвращая клиенту корректный код 409 Conflict. Это решило проблему овербукинга на 100%.

3. Организация процесса разработки

- **Команда:** Проект выполнялся в паре.
 - *Студент А (Backend):* Проектирование БД PostgreSQL, написание эндпоинтов на FastAPI, настройка Docker.
 - *Студент Б (Frontend):* Верстка интерфейса в Tailwind CSS, написание React-компонентов, связь фронтенда с API через Axios.
- **Процессы:** Задачи велись на Канбан-доске в Trello. Код хранился на GitHub. Использовалась базовая модель ветвления: ветка main для стабильного кода, разработка велась в ветках feature/auth, feature/booking с последующим слиянием через Pull Requests.

4. Рефлексия и приобретенный опыт

Если бы мы начинали проект заново, мы бы уделили больше времени начальному проектированию базы данных. Из-за ошибки в структуре таблиц нам пришлось на середине проекта писать сложные скрипты миграции (Alembic) для изменения типа связи с один-к-одному на один-ко-многим. Тем не менее, проект дал огромный опыт в понимании работы асинхронного Python и принципов работы реляционных БД под высокой нагрузкой.

5. План развития (Roadmap на следующие релизы)

1. **Q3 2026:** Замена платежной заглушки на реальную интеграцию с эквайрингом через webhook-уведомления.
2. **Q3 2026:** Добавление ролевой модели "Охранник" (проверка QR-кодов бронирований на входе через мобильный сканер).
3. **Q4 2026:** Внедрение системы лояльности (накопление бонусов за часы бронирования).